

대기열·마감시한 MEC Offloading을 위한 분산 DRL

Deep Reinforcement Learning for Task Offloading in Mobile Edge Computing Systems

Ming Tang and Vincent W. S. Wong (2022)

새 task의 위치 선택이 이전 task의 대기과 미래 edge load에 의해 바뀌는 상황을 다룬다

왜 한 slot 안에 끝난다고 가정할 수 없을까

새 task의 지연은 자기 선택만이 아니라 앞선 task의 service-transfer와 다른 device의 edge load에 영향을 받는다

01 새 task slot 시작에 확률적으로 도착



02 위치 선택 local 또는 edge 하나를 선택



03 FIFO 대기 앞선 task가 남아 있으면 기다림



04 여러 slot service 처리-전송은 이어질 수 있음



05 완료 또는 drop deadline 끝에서 아직 미완료면 drop

이 논문의 난점은 multi-slot queue가 action의 결과와 feedback 시점을 밀어낸다는 것이다

먼저 고정해야 하는 모델 경계

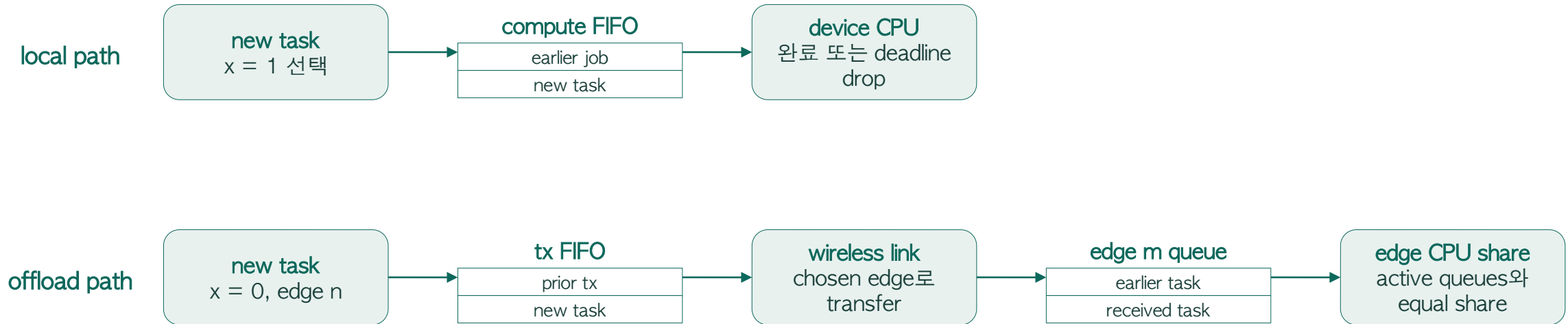
이 텍스트의 algorithm 설명은 논문이 둔 task·wireless·edge scheduling 가정 안에서만 성립한다

항목	논문의 모델	설명 경계
task	non-divisible, deadline 보유	분할·병렬 edge 실행 아님
device queue	local compute와 transmission FIFO	새 task는 둘 중 하나만 선택
wireless	orthogonal channel, constant rate	error·interference 미포함
edge CPU	device별 FIFO queue의 equal-share GPS	우선순위 scheduler 아님
decision	device가 local observation으로 선택	runtime central selector 아님

결과를 읽기 전에 task non-divisibility와 queue service rule을 함께 확인해야 한다

새 task가 만나는 세 대기 지점

local 선택은 device compute queue로, offload 선택은 transmission queue와 선택 edge의 device queue로 간다



offload path는 transmission wait만이 아니라 edge-side queue와 CPU share까지 포함한다

slot 순서: enqueue 뒤 service, 다음 job은 다음 slot

완료된 job이 있어도 같은 slot 안에서 뒤 job을 즉시 시작시키지 않는 규칙이 waiting calculation의 기준이다

01 slot 시작 새 task arrival과 state 관측



02 action queue와 edge destination 결정



03 enqueue 선택한 FIFO에 task 추가



04 service 현재 job이 transfer 또는 process



05 다음 slot 완료 뒤 다음 job이 시작

‘완료 slot’과 ‘다음 job의 start slot’을 구분해야 queue waiting을 한 칸 틀리지 않는다

local FIFO wait은 completion-drop slot으로 계산한다

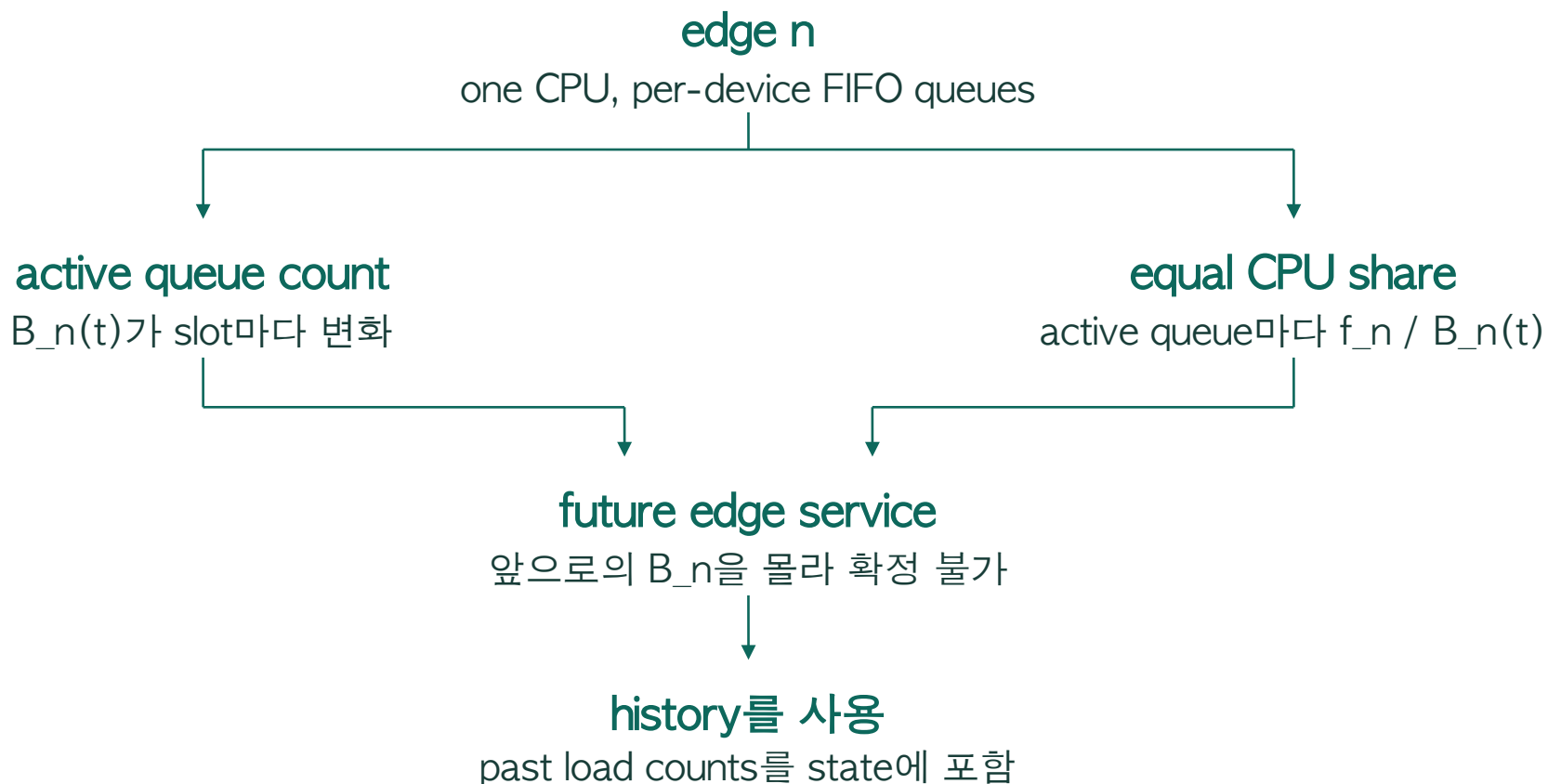
source Eq. 2는 이전 local jobs가 끝나는 가장 늦은 slot을 사용한다

상황	계산	뜻
new task at slot t	$\text{latest earlier } l_{\text{comp}} - t + 1$	아직 남은 FIFO wait
earlier job ends at 5	task arrives at 3	$\text{wait} = 5 - 3 + 1 = 3$
local service	$\text{ceil}(\text{size} / \text{per-slot capacity})$	필요한 service slot 수
deadline	$\text{min}(\text{service end}, t + \text{tau} - 1)$	완료 또는 drop slot

deadline은 대기 뒤의 service가 끝나는 시점과 경쟁하며, 늦으면 task를 drop시킨다

edge 처리 지연은 왜 action 순간에 확정되지 않는가

edge n 의 active queue 수가 slot마다 바뀌면 task m 이 받는 CPU share도 바뀐다



state에는 관측 가능한 것과 예측해야 할 것을 분리한다

device는 자신의 queue state와 edge load history를 쓰지만, future edge queue waiting은 모른다

state component	어떻게 얻나	의미
task size	새 arrival에서 local observation	이번 task의 service량
local-tx wait	device FIFO completion history	이미 생긴 device backlog
내 edge queue 벡터	전송·완료·drop 기록	각 edge에서 남은 own bits
부하 이력 $H(t)$	edge의 active-count broadcast	최근 edge congestion pattern
future edge wait	결정 시 알 수 없음	state에 직접 넣지 않음

LSTM의 입력은 미래 정답이 아니라 공개된 과거 active-queue count sequence다

action은 'local인가'와 '어느 edge인가'의 결합이다

새 task가 있을 때 action $a_m(t)$ 는 binary x 와 destination vector y 로 구성된다

선택	x	y 의 조건	queue 결과
local	1	모든 $y_{m,n} = 0$	device computation FIFO
edge n 전송	0	선택 n 만 $y_{m,n} = 1$	device tx FIFO $\rightarrow$ edge n
선택 조건	$x = 0$	sum over n of $y_{m,n} = 1$	edge 하나만 선택
허용 안 됨	—	여러 y 가 동시에 1	task splitting 아님

binary offloading은 edge 하나를 고르는 whole-task placement이며 splitting이 아니다

reward가 아니라 discounted cost를 최소화한다

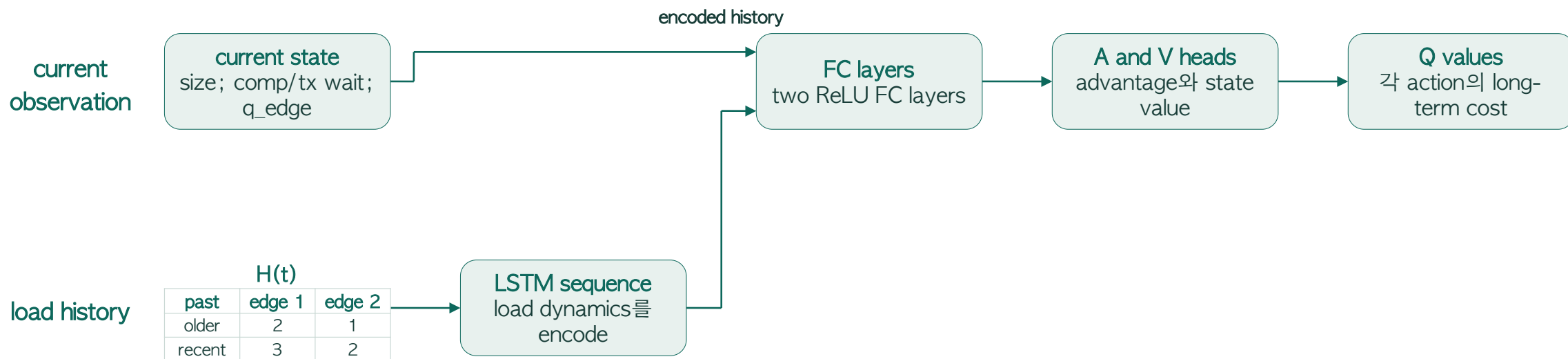
이 논문에서 Q가 작다는 것은 state-action pair의 expected long-term cost가 작다는 뜻이다

01	processed task	arrival부터 completion까지 delay
↓		
02	dropped task	positive fixed penalty C
↓		
03	no arrival	cost 0
↓		
04	discount	future cost에 gamma 적용
↓		
05	policy	expected long-term cost 최소화

processed-task delay뿐 아니라 deadline drop penalty까지 action cost에 넣는다

current feature와 load history의 두 input path

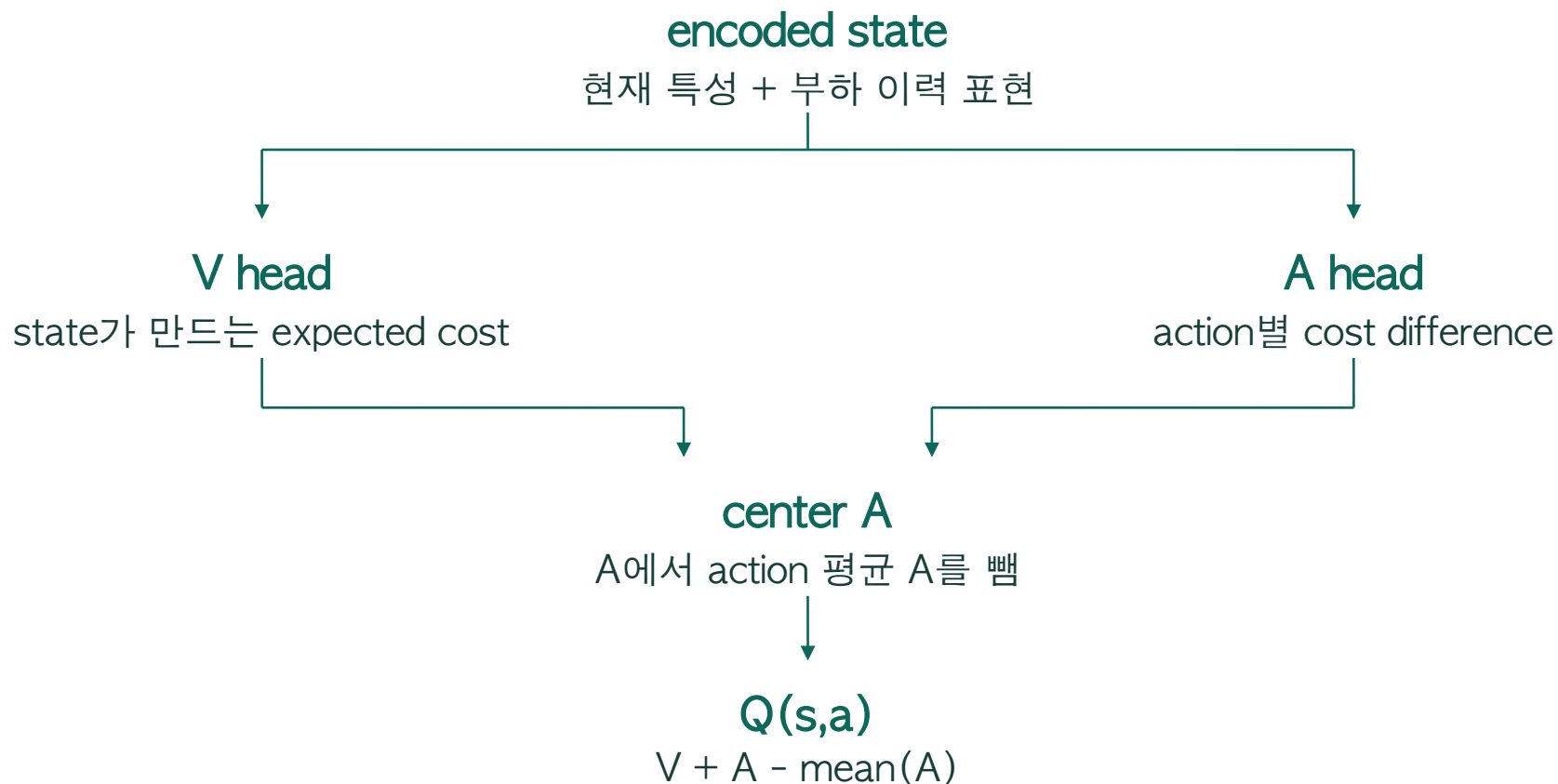
LSTM은 $H(t)$ 의 temporal pattern을, FC path는 현재 task-queue feature를 Q estimation에 연결한다



history path는 future load 정답이 아니라 past load variation을 Q estimator에 전달한다

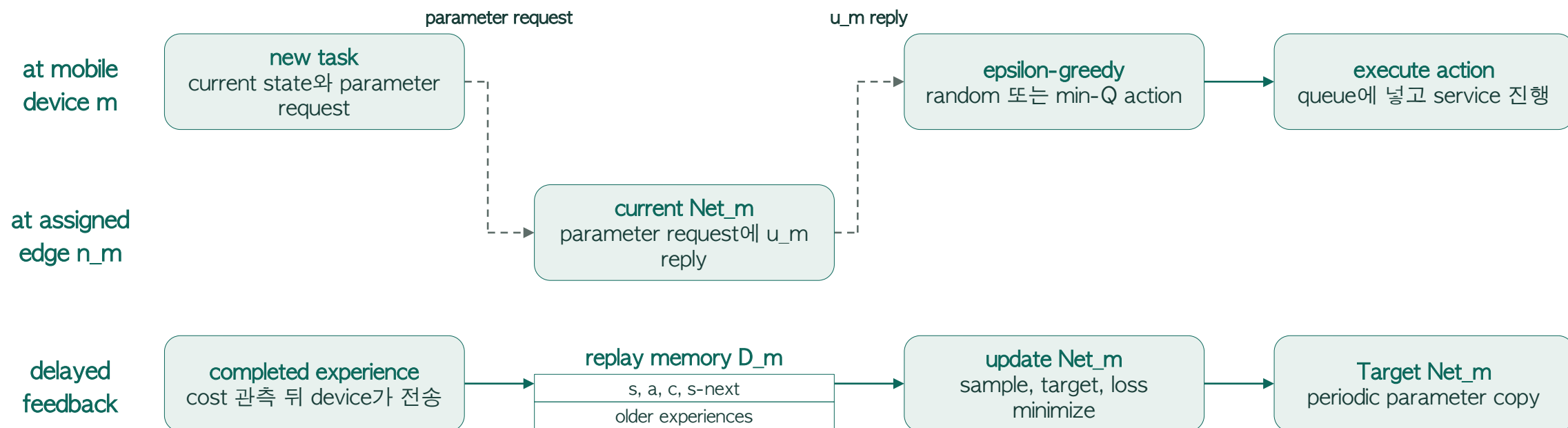
dueling DQN: state 자체의 비용과 action 차이를 나눈다

Q는 state value V 와 action advantage A 를 결합하되, A 의 평균을 빼 identifiability를 맞춘다



runtime action과 edge-assisted learning은 분리된다

device는 current Net parameter로 행동하고, assigned edge node는 device별 memory와 networks를 training한다



action은 decentralized지만, replay와 learning computation은 edge가 보조한다

multi-slot task의 delayed cost feedback

논문은 task가 실제로 processed 또는 dropped된 slot에 그 task의 experience를 보낸다

01	slot t	state s_t 에서 action a_t 선택
↓		
02	slot $t+1$	next state s_{t+1} 관측
↓		
03	service 계속	task t 의 delay·drop은 미확정 가능
↓		
04	finish slot	processed 또는 dropped task 확인
↓		
05	experience	s_t, a_t, c_t, s_{t+1} 전송

next state와 cost observation 시점이 분리되므로 completed-task set을 따로 만든다

double DQN target: action 선택과 평가를 분리한다

evaluation Net이 next action을 고르고, Target Net이 그 action의 discounted future cost를 평가한다

01	replay sample	s_i, a_i, c_i, s_{i+1}
↓		
02	choose next action	Net에서 min-Q action 선택
↓		
03	evaluate target	Target Net에서 next-action Q
↓		
04	target cost	$c_i + \gamma \times \text{target Q}$
↓		
05	periodic copy	Target Net $\leftarrow$ Net parameters

double DQN은 target에서 next-action choice와 value evaluation을 분리한다

구성 예시: local queue만으로 기다림을 다시 계산해 보자

아래 숫자는 source result가 아닌, Eq. 2-3의 queue accounting을 확인하기 위한 구성 예시다

slot / 조건	계산	결과
prior local job	completion slot = 5	queue가 5까지 점유
new task	arrival $t = 3$	local 선택
waiting	$\max(5 - 3 + 1, 0)$	3 slots
service	2 slots, deadline end slot 8	unbounded end = 7
cost	completion at 7	$\text{delay} = 7 - 3 + 1 = 5$

task는 deadline 전 slot 7에 끝나 delay cost를 받고, edge future wait은 예측하지 않는다

비교 방법은 무엇을 다르게 가정하는가

source baselines는 같은 metric 아래 비교되지만, feedback·capacity model과 queue assumptions가 완전히 같지는 않다

method	선택 방식	논문이 밝힌 차이
No Offl.	언제나 local	edge capacity를 사용하지 않음
R. Offl.	random offloading	load-aware policy 아님
PGOA	potential-game best response	각 task가 one slot 내 끝난다고 가정
ULOOF	historical capacity estimation	다른 load-estimation route
proposed DRL	history-based Q learning	multi-slot queue·delayed cost를 모델링

비교 핵심은 DRL 이름보다 multi-slot queue와 unknown edge load를 다루는 방식이다

reported result를 읽기 위한 simulation 조건

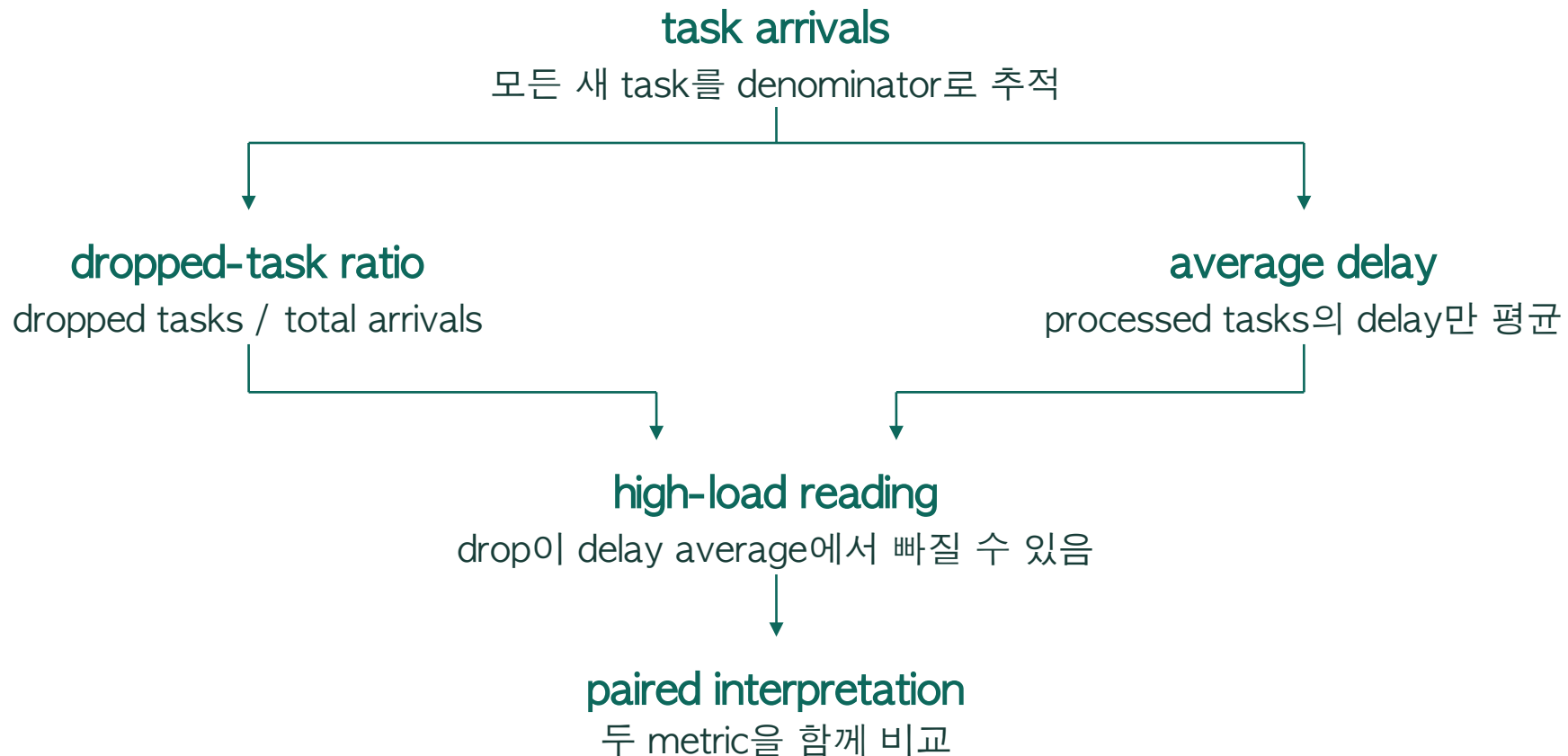
성능 수치는 50 devices·5 edges와 stationary transition/cost function 아래의 online training 결과다

parameter	source setting	해석
system size	50 mobile devices, 5 edge nodes	임의 규모의 scaling proof 아님
slot / deadline	0.1 s / 10 slots = 1 s	deadline sweep은 별도 figure
CPU / link	2.5 GHz device, 41.8 GHz edge, 14 Mbps	fixed simulation parameters
learning	batch 16, lr 0.001, gamma 0.9	exploration 1 → 0.01
environment	stationary transition and cost	non-stationarity field test 아님

Table 1과 Fig. 4는 simulation condition이며 production hardware guarantee가 아니다

drop ratio와 average delay는 반드시 함께 읽는다

source average delay는 processed tasks만 평균내므로, drop이 늘면 average delay가 내려갈 수도 있다



source가 보고한 조건부 결과

다음 수치는 figure의 sweep에서 나온 observation이며, 모든 MEC topology의 보편적 보장은 아니다

sweep / condition	reported observation	읽을 때 남길 조건
arrival p 0.1 $\rightarrow$ 0.5	DRL drop < 0.2 ; others > 0.5	Fig. 5 simulation
deadline 0.6 s	drop 65.8–79.3% lower	listed baselines
deadline ≥ 1.4 s	DRL delay near 0.54 s	others > 0.84 s
80 devices	DRL drop < 0.05	Fig. 7 sweep
150 devices	delay 9.0% lower	processed-task delay

reported benefit은 tight deadline·high modeled load의 stated simulation에 한정된다

논문이 보여 주지 않은 것까지 확장하지 않는다

DRL-based Q approximation과 simulation evidence는 valuable하지만, source가 보장하지 않은 범위를 따로 남겨야 한다

source establishes

multi-slot FIFO simulation

history-input LSTM-DQN

fixed wireless and GPS comparison

edge-assisted training

cost extension mention

not established

DRL convergence proof

exact future edge-delay observation

error-interference field deployment

no-overhead unlimited scaling

energy-optimality result

convergence, wireless realism, parameter overhead, non-stationarity는 여전히 열린 범위다

핵심 정리

이 논문은 offloading action 하나를 queue, deadline, hidden edge load, delayed feedback과 함께 다시 정의한다

- whole task는 local 또는 one edge로 가며 queue에서 대기한다
- state는 known local queue와 broadcast edge-load history를 구분한다
- LSTM·dueling DQN·double DQN은 history, Q decomposition, target 역할을 맡는다
- device action은 decentralized이고 edge node는 replay·training을 보조한다
- drop ratio와 processed-task delay를 함께 읽어야 result를 오독하지 않는다

‘어디로 보낼까’는 queue-aware delayed-feedback decision으로 이해해야 한다